

ServiceNow Business Rules

A Complete Quick-Reference Guide

From Fundamentals to Real-World Implementation: Execution Types, Best Practices, Common Mistakes, and Hands-On Scenarios for CSA & CAD Learners

Prepared as a study and reference resource for ServiceNow developers and administrators

Table of Contents

1. What Are Business Rules?
2. Where Business Rules Fit in the ServiceNow Architecture
3. The Four Execution Types
4. Before vs After vs Async vs Display — Comparison Table
5. Anatomy of a Business Rule Script
6. Best Practices
7. Common Mistakes to Avoid
8. Execution Order and Run Order Considerations
9. Why Business Rules Matter
10. Real-World Scenarios
 - 10.1 Automatic Ticket Assignment
 - 10.2 Approval Validation
 - 10.3 SLA Updates
 - 10.4 Notification Triggers
11. Quick Revision Checklist

1. What Are Business Rules?

Business Rules are server-side scripts in ServiceNow that execute automatically whenever a record is inserted, updated, deleted, or queried on a table. They are one of the most fundamental building blocks of platform customization, allowing administrators and developers to enforce business logic, maintain data consistency, and automate workflows without requiring manual intervention from end users.

Because Business Rules run on the server, they execute regardless of which interface (Service Portal, Now Mobile, Now Experience UI, or even REST API calls) triggers the database operation. This makes them more reliable for enforcing logic than client-side scripts, which only run in a browser context.

Typical responsibilities handled by Business Rules include:

- Validating field values before a record is saved
- Automatically populating or recalculating fields
- Updating related records when a parent record changes
- Triggering notifications or workflow events
- Performing background or scheduled-style processing through Async execution

2. Where Business Rules Fit in the ServiceNow Architecture

ServiceNow processes a database operation through several layers. Understanding where Business Rules sit in that pipeline helps explain why execution timing (Before/After/Async/Display) matters so much.

Layer	Description
Client Scripts	Run in the browser before the request reaches the server (onLoad, onChange, onSubmit).
Access Control (ACL)	Evaluates whether the user is permitted to read/write the record.
Before Business Rules	Execute on the server before the record is written to the database.
Database Operation	The actual insert, update, or delete is committed to the database.
After Business Rules	Execute once the database operation has completed successfully.
Async Business Rules	Queued to the system scheduler and executed independently, after the transaction.

Display Business Rules are the exception to this pipeline — they don't act on the database operation itself, but instead run when a form is loading, to push server-side data into the `g_scratchpad` object for client-side use.

3. The Four Execution Types

3.1 Before

Before Business Rules run prior to the record being saved to the database. They are used to validate incoming data or modify field values before they are committed. Because they run before the save, any changes made to **current** fields are automatically included in the same database write — there is no need to call `current.update()`.

```
// Example: Before Business Rule
// Table: incident | When: before | Insert/Update

if (current.priority == 1) {
  current.urgency = 1;
  current.impact = 1;
}
```

3.2 After

After Business Rules execute once the record has already been written to the database. They are commonly used to update related records, since the triggering record's `sys_id` and saved values are guaranteed to exist at this point.

```
// Example: After Business Rule
// Table: incident | When: after | Update

if (current.state == 6) {
  var gr = new GlideRecord('problem');
  if (gr.get(current.problem_id)) {
    gr.work_notes = 'Linked incident resolved.';
    gr.update();
  }
}
```

3.3 Async

Async Business Rules are queued and processed in the background by the ServiceNow scheduler, rather than running inline with the user's transaction. This prevents long-running logic — such as outbound integrations, heavy calculations, or bulk record updates — from delaying the save and degrading the user's perceived performance.

```
// Example: Async Business Rule
// Table: incident | When: async | Insert

var gr = new GlideRecord('sys_email');
gr.initialize();
gr.subject = 'New Incident Created: ' + current.number;
gr.insert();
```

3.4 Display

Display Business Rules run when a form is loading, before the page is rendered in the browser. They are used to pass server-side data (such as values requiring a GlideRecord query) to the client through the `g_scratchpad` object, since client scripts cannot directly query the database.

```
// Example: Display Business Rule
// Table: incident | When: display

g_scratchpad.slaBreached = current.sla_due < new GlideDateTime();
```

4. Before vs After vs Async vs Display — Comparison

Type	Runs	Can modify current?	Typical Use
Before	Before DB write	Yes (auto-saved)	Validation, field defaults
After	After DB write	Yes (needs <code>current.update()</code>)	Update related records
Async	Background, post-transaction	Yes (needs <code>current.update()</code>)	Long-running / non-blocking tasks
Display	On form load	No (use <code>g_scratchpad</code>)	Pass server data to client

Note: In After and Async rules, modifying current does not automatically save — you must explicitly call `current.update()` if a further write is required. Avoid calling `current.update()` inside a Before rule, since it triggers a redundant duplicate save and can cause infinite recursion in some configurations.

5. Anatomy of a Business Rule Script

Every Business Rule, regardless of execution type, shares a common structural pattern. Understanding these parts makes it easier to read, debug, and write new rules confidently.

- **current** — a GlideRecord representing the record being processed.
- **previous** — (available in After/Async on update) a GlideRecord representing the record's prior state.
- **(current, previous)** — the standard function signature used by the script's scope.
- **Condition field** — an optional filter condition evaluated before the script runs, used to limit unnecessary execution.
- **Order field** — a number controlling the sequence in which multiple rules on the same table/event run.

```
(function executeRule(current, previous /*null when async*/) {  
  
  // Business logic goes here  
  if (current.state.changesTo(6)) {  
    current.resolved_at = new GlideDateTime();  
  }  
  
})(current, previous);
```

6. Best Practices

Choose the correct execution timing

Match the execution type to the goal: validate with Before, propagate with After, offload heavy work to Async, and feed the form with Display.

Add conditions so rules run only when necessary

Use the Condition field (or an early-exit check in the script) so the rule's logic only executes when it is actually relevant, reducing unnecessary processing on every transaction.

Keep scripts short, reusable, and well documented

Favor calling a Script Include for complex logic over writing large inline scripts. Add comments explaining intent, not just mechanics.

Prefer Async for long-running operations

Anything involving external integrations, large GlideRecord queries, or bulk updates should be moved to Async so the user's save isn't delayed.

Test execution order when multiple rules interact

When several Business Rules act on the same table and event, verify the Order field to ensure they fire in the intended sequence and don't conflict.

7. Common Mistakes to Avoid

Avoid using `current.update()` inside Business Rules

Calling `current.update()` inside a Before rule causes a redundant save and can trigger the rule to fire again, leading to infinite loops in poorly guarded scripts. In After/Async rules it is sometimes necessary, but should be used sparingly and only when truly required.

Don't duplicate logic across multiple rules

Repeating the same validation or calculation in several Business Rules makes the system harder to maintain and increases the risk of inconsistent behavior. Centralize shared logic in a Script Include.

Don't perform unnecessary database queries

Every `GlideRecord` query inside a Business Rule adds processing time. Query only when needed, and use conditions to skip the rule entirely when its logic doesn't apply.

Don't use Display rules to modify stored data

Display Business Rules are meant to prepare data for the client via `g_scratchpad` — they should never be used to write or update records, since this breaks the intended separation of concerns and can produce unpredictable results.

8. Execution Order and Run Order Considerations

When multiple Business Rules are defined for the same table and the same execution event (for example, several Before/Update rules on the Incident table), ServiceNow executes them in ascending order based on the Order field, which defaults to 100.

- Lower order values run first (e.g., a rule with order 50 runs before one with order 100).
- Rules with the same order value run in an indeterminate sequence — always assign distinct order numbers when sequence matters.
- Before rules complete fully (including all of them, in order) before the database write occurs.
- After rules run only once the database write has succeeded, again respecting their own order sequence.
- Use the system log or breakpoints in a sub-production instance to confirm actual execution order during testing.

Best practice: leave gaps between order values (e.g., 100, 200, 300) when initially designing a set of rules, so new rules can be inserted between existing ones later without renumbering everything.

9. Why Business Rules Matter

Well-designed Business Rules are central to building scalable, maintainable ServiceNow applications. Their impact extends across several dimensions of platform quality:

- **Performance** — properly scoped, conditioned rules avoid unnecessary processing on every transaction.
- **Data integrity** — validation logic prevents bad data from ever reaching the database.
- **Reduced manual work** — automated field population and related-record updates remove repetitive tasks from end users.
- **Scalability** — modular, well-documented rules are easier to extend as an application grows in complexity.
- **Consistency across channels** — because rules run server-side, the same logic applies whether a record is changed via the UI, mobile app, or API.

10. Real-World Scenarios

The scenarios below are good practice exercises for reinforcing Business Rule concepts in a Personal Developer Instance (PDI).

10.1 Automatic Ticket Assignment

Goal: When an incident is created with a specific category, automatically assign it to the appropriate assignment group.

```
// Table: incident | When: before | Insert

if (current.category == 'network') {
current.assignment_group = '<sys_id of Network Team>';
} else if (current.category == 'hardware') {
current.assignment_group = '<sys_id of Hardware Team>';
}
```

10.2 Approval Validation

Goal: Prevent a change request from being moved to 'Implement' state unless all required approvals have been completed.

```
// Table: change_request | When: before | Update

if (current.state.changesTo('implement')) {
if (current.approval != 'approved') {
gs.addErrorMessage('Change cannot proceed without approval.');
```

10.3 SLA Updates

Goal: When an incident's priority changes, recalculate and update the associated SLA due date in an After rule so the change is reflected once the record is committed.

```
// Table: incident | When: after | Update

if (current.priority.changes()) {
var sla = new GlideRecord('task_sla');
sla.addQuery('task', current.sys_id);
sla.query();
while (sla.next()) {
sla.planned_end_time = calculateNewDueDate(current.priority);
sla.update();
}
}
```

10.4 Notification Triggers

Goal: Send an email notification to the assigned engineer whenever a high-priority incident is assigned to them, without delaying the save (use Async).

```
// Table: incident | When: async | Insert/Update

if (current.priority == 1 && current.assigned_to.changes()) {
  var email = new GlideEmailOutbound();
  email.setSubject('High Priority Incident Assigned: ' + current.number);
  email.setBody('You have been assigned a P1 incident. Please review immediately.');
```

email.addAddress('to', current.assigned_to.email);

email.save();

}

11. Quick Revision Checklist

#	Checklist Item	Done
1	I can explain the difference between Before, After, Async, and Display rules	☐
2	I know when to use Async instead of After	☐
3	I understand why <code>current.update()</code> should be avoided in Before rules	☐
4	I can explain how the Order field affects rule execution sequence	☐
5	I know that Display rules use <code>g_scratchpad</code> and must not modify stored data	☐
6	I can write a conditioned rule to avoid unnecessary execution	☐
7	I can identify when logic should move to a Script Include instead	☐
8	I have practiced at least one real-world scenario in a PDI	☐

Keep building real-world scenarios — automatic ticket assignment, approval validation, SLA updates, and notification triggers — in a Personal Developer Instance to reinforce these concepts before sitting the CSA or CAD exam.